

Building and Deploying the Resume Analyser

A Step-by-Step Guide for Students

Welcome! This document will walk you through the entire process of how this Resume Analyser project was built from scratch and how it was deployed to the internet.

1. The Technology Stack

Before writing any code, we selected modern, industry-standard tools to build this web application. Here is what we used:

- **React (v19)**: A popular JavaScript library developed by Facebook for building user interfaces. It allows us to create reusable UI components.
- **TypeScript**: A superset of JavaScript that adds static typing. It helps catch errors early during development and provides better autocomplete in your code editor.
- **Vite**: A blazing fast build tool and development server. It replaces older tools like Create React App or Webpack, making the development experience much faster.
- **Tailwind CSS (v4)**: A utility-first CSS framework that lets you style your application directly inside your HTML/JSX without writing separate CSS files.
- **Framer Motion**: A powerful animation library for React that makes it easy to create smooth, beautiful animations.
- **Lucide React**: A beautiful, open-source icon library.
- **PDF.js**: A tool by Mozilla used to parse and read the text inside uploaded PDF resumes directly in the browser.
- **Surge (surge.sh)**: A free static web publishing platform that allows you to deploy web applications directly from your terminal.

2. Step-by-Step Building Process

Step 2.1: Initializing the Project

We started by creating a new React project using Vite. You can do this by running the following command in your terminal:

```
npm create vite@latest resume-analyser -- --template react-ts
```

This command creates a new folder called `resume-analyser` with all the base configurations for React and TypeScript.

Step 2.2: Installing Dependencies

After navigating into the project folder (`cd resume-analyser`), we installed the required libraries:

```
# Install core dependencies
npm install

# Install UI and utility libraries
npm install tailwindcss @tailwindcss/vite lucide-react framer-motion axios pdfjs-dist
```

Step 2.3: Project Structure

We organized our code into a clear folder structure inside the `src` directory:

- `src/components/`: Contains our React UI building blocks like `ResumeInput.tsx` (where users upload PDFs) and `ResultsDashboard.tsx` (where the analysis is shown).
- `src/utils/`: Contains helper functions, specifically `pdfParser.ts` which handles extracting text from the PDF file.
- `src/App.tsx`: The main container that brings all components together.

Step 2.4: Developing the Features

1. **PDF Parsing**: We used `pdfjs-dist` to read the uploaded PDF file byte-by-byte and convert it into a plain text string.
2. **User Interface**: We built forms and buttons using standard HTML elements styled beautifully with Tailwind CSS. For example, instead of writing `margin-top: 10px; color: blue;`, we simply added classes like `mt-2 text-blue-500`.
3. **State Management**: We used React's `useState` hook to keep track of whether the application is loading, what errors exist, and the final parsed resume text.

Step 2.5: Running the Project Locally

During development, we tested the code on our own computer by running:

```
npm run dev
```

This started a local server (usually at `http://localhost:5173`) where every code change automatically refreshed the browser.

3. The Deployment Process

Once the code was working perfectly on our local computer, we needed to put it on the internet so anyone could access it. Here is exactly how we did it.

Step 3.1: Building for Production

Web browsers cannot natively run TypeScript files (`.tsx`). They only understand plain HTML, CSS, and JavaScript. We told Vite to translate and bundle our code into highly optimized, minified files by running:

```
npm run build
```

This command successfully generated a `dist` (distribution) folder containing the final, ready-to-publish website files.

Step 3.2: Deploying with Surge

To host the `dist` folder on the internet for free, we used **Surge.sh**.

1. We used `npx` (Node Package Execute) to run Surge directly on our `dist` folder:

```
npx surge ./dist
```

2. Surge prompted us to create an account by simply entering an **email** and a **password**.
3. It then asked us for a **domain name**. We provided a unique name: `resume-analyser-zuhair123.surge.sh`.
4. Surge securely uploaded the 6 optimized files (HTML, JS, CSS, and assets) from our `dist` folder to their globally distributed servers.

Within a few seconds, Surge provided the live link: <https://resume-analyser-zuhair123.surge.sh>

Conclusion

By combining Vite for fast development, React for UI components, Tailwind for styling, and Surge for instant deployment, we built and launched a modern web application incredibly quickly. As a student, mastering this exact workflow will give you the superpower to turn your ideas into live products in a matter of minutes!